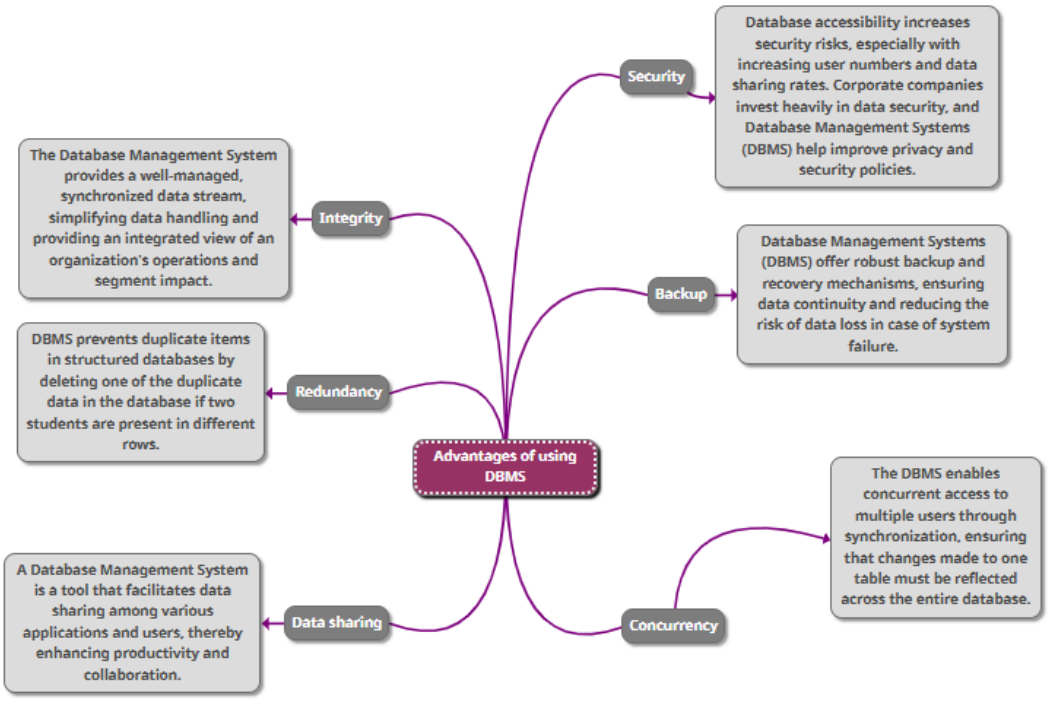
1. Comparison Assignment:

	Flat File Systems	Relational Databases
Structure	- Simple, unstructured format with records stored in plain text files. - Stores all data in a single table.	- Highly structured format with tables, rows, and columns. - Provides flexible, ongoing projections using updated data.
Data Redundancy	it has a lot of redundant data, and somebody will have to enter and maintain that data.	does not have redundancy and is much easier to maintain, it is often much faster than a flat file, especially when lots of data is involved.
Relationships	No relationships between data.	Relationships established through foreign keys.
Example usage	- Simple Contact Lists - Data Import/Export - Spreadsheets - Small-scale applications	- Government Records - University Administration - banking systems - E-commerce platforms
Drawbacks	- Data Inconsistency (Difficult to update data consistently) - Data Redundancy	- difficult to set up and maintain. - Increased cost - Frequency upgrade/Replacement cycles

2. DBMS Advantages Mind Map:



3. Roles in a Database System:

	Roles
System Analyst	System Analyst bridges business requirements and technical solutions by analyzing requirements, defining system specifications, working with stakeholders, and collaborating with designers and developers to meet user needs.
Database Designer	Database Designer creates database models, defines tables, relationships, keys, constraints, and ensures design supports business requirements, optimizing structure for scalability and efficiency.
Database Developer	Database developer implements and maintains databases, writing SQL queries, developing data import/export processes, ensuring data accuracy, performance, and security, and working closely with database designers and application developers.
DBA (Admin)	DBA maintains database system health, performance, and security by installing, configuring, upgrading software, performing backups, disaster planning, monitoring performance, and managing user access and security settings.
Application Developer	Application developer creates software applications that interact with databases, integrates application logic with queries, ensures efficient data access, and collaborates with DBAs and developers for optimal performance.
BI Developer	The BI Developer transforms data into actionable insights for business decision-making, designing dashboards, reports, ETL pipelines, working with data warehouses, analytics tools, and collaborating with stakeholders.

4. Additional Research Topics:

4.1. Types of Databases:

4.1.1. Relational vs Non-Relational:

Relational databases, like MySQL, PostgreSQL, and Oracle, have strict schemas and relationships, suitable for financial systems, CRM applications, and ERP systems. They have strong ACID compliance and are suitable for complex joins and structured data.

Non-relational databases, like MongoDB, Cassandra, and Redis, offer high scalability and flexibility.

4.1.2. Centralized vs Distributed vs Cloud Databases:

Centralized databases are managed on a single server, offering simplicity and security.

Distributed databases distribute data across multiple locations, improving reliability and scalability.

Cloud databases, hosted on platforms like AWS, Azure, or Google Cloud, offer scalability on demand, reduced infrastructure management, high availability, and disaster recovery, suitable for SaaS applications and scalable web applications.

4.1.3. Use case example:

- **Relational (MySQL):** Banking systems where data integrity is critical.
- **Non-Relational (MongoDB):** Content-heavy websites needing flexible schemas (e.g., blogs, e-commerce catalogs).
- **Centralized Database:** Inventory management for a small retail store.
- **Distributed Database (Cassandra):** IoT platforms requiring distributed data storage for real-time analytics.
- **Cloud Database (Amazon RDS):** Scalable backend for a mobile app with fluctuating user traffic.

4.2. Cloud Storage and Databases:

4.2.1. What is Cloud Storage and how does it relate to database?

Cloud storage is the storage of data on remote servers, maintained by providers like AWS, Microsoft Azure, and Google Cloud Platform. It is typically used for unstructured data and backups, while cloud databases, hosted in the cloud, manage structured or semi-structured data and offer querying, indexing, and transactional features.

4.2.2. Advantages and Disadvantages of using cloud-based database:

Advantages: Cloud-based databases offer scalability, high availability, cost-efficiency, automatic updates, global accessibility, and security features. They provide redundancy, failover support, and compliance with standards like GDPR and HIPAA, eliminating the need for upfront hardware investments.

Disadvantages: Cloud-based databases have several disadvantages, including internet dependency, limited control, data privacy concerns, higher latency, and increased costs over time due to improper management and usage.

4.3. Database Engines and Languages:

4.3.1. What is Database Engine?

A Database Engine is a crucial software component in a database system, responsible for data storage, retrieval, manipulation, indexing, and security management.

4.3.2. Examples:

Database Engine	DBMS	Primary Query Language
SQL Server	Microsoft SQL Server	T-SQL (Transact-SQL)
MySQL	MySQL (Oracle-owned)	ANSI SQL + MySQL Extensions
Oracle	Oracle Database	PL/SQL (Procedural SQL)
PostgreSQL	PostgreSQL	ANSI SQL + PL/pgSQL

4.3.3. What language do they use?

- **ANSI SQL**: The standard SQL language used as a baseline across most engines.
- **T-SQL**: Microsoft's extension of SQL with procedural programming features (used in SQL Server).
- **PL/SQL**: Oracle's procedural extension of SQL, allowing loops, conditions, and variables.
- **PL/pgSQL**: PostgreSQL’s procedural language, similar to PL/SQL.

4.3.4. Is there a relationship between the engine and the language?

Engines support SQL dialects aligned with ANSI standard, each with proprietary extensions for procedures, triggers, and error handling. T-SQL runs on SQL Server, while PL/SQL is exclusive to Oracle Database.

4.3.5. Can one language work across different engines?

Language compatibility across engines depends on syntax and feature differences, with basic ANSI SQL queries being portable. Migration tools and ORMs can help abstract these differences.

4.4. Can we transfer a Database between engines?

4.4.1. Database migration between engines like SQL Server, MySQL, or Oracle involves complex transformation and compatibility work due to differences in data handling, queries, and logic.

Common Engine-to-Engine Migration Examples:

From	To	Migration Tools
SQL Server	MySQL	AWS DMS, MySQL Workbench, SSMA for MySQL
Oracle	PostgreSQL	ora2pg, AWS DMS, EDB Migration Toolkit
SQL Server	PostgreSQL	SQL Server Migration Assistant (SSMA), pgloader
Oracle	MySQL	Oracle SQL Developer, MySQL Shell

4.4.2. Challenges of engine-to-engine migration:

Engine-to-engine migration faces challenges such as data type incompatibilities, SQL dialect differences, schema differences, performance optimization, application dependency, and data integrity and migration errors. These issues require rewriting logic in the target engine's language, tuning query execution plans, and addressing application dependencies.

4.4.3. What should we consider before transferring?

Aspect	Consideration
Data Types	Ensure each data type in the source has a valid equivalent in the target system.
Stored Procedures	Rewrite in the new engine’s procedural language.
Triggers and Functions	Redesign logic to match the target engine's capabilities.
Views	Check for compatibility and rewrite complex queries if needed.
Indexes and Constraints	Match indexing logic and ensure all constraints are replicated correctly.
Security and Roles	Re-map users, roles, and permissions appropriately.
Character Encoding	Ensure consistency (e.g., UTF-8 vs Latin1).
Dependencies	Review application-side queries or tools dependent on the original engine syntax.

4.5. Logical vs. Physical Schema:

4.5.1. What is Logical Schema in database design?

A Logical Schema is an abstract database structure that defines data storage, relationships, and rules, focusing on business requirements and platform-independent, excluding specific software and ER diagrams.

4.5.2. What is the Physical Schema?

A Physical Schema is a specific DBMS implementation that includes table definitions, column data types, indexes, partitions, and storage paths, focusing on performance, storage, and access paths.

4.5.3. Difference Between Logical and Physical Schema:

Aspect	Logical Schema	Physical Schema
Purpose	Conceptual design based on business needs	Implementation and performance optimization
Platform-Dependent	No	Yes (DBMS-specific)
Focus	Data structure and relationships	Storage details and access mechanisms
Components	Entities, attributes, relationships	Tables, columns, data types, indexes, storage paths

4.5.4. Why is it important to understand both?

Understanding both logical and physical schemas is crucial for database design and implementation, particularly during migration, scaling, and optimization, as they ensure alignment with business rules and requirements.

4.5.5. Example:

Employee Entity

Logical Schema:

Entity: Employee

Attributes:

- EmployeeID (PK)
- FullName
- HireDate
- Department
- Manager (self-referencing relationship)

Physical Schema (SQL Server – T-SQL):

```
CREATE TABLE Employee (
    EmployeeID INT PRIMARY KEY,
    FullName NVARCHAR(100),
    HireDate DATE,
    Department NVARCHAR(50),
    ManagerID INT FOREIGN KEY REFERENCES Employee(EmployeeID)
);
```